React Native Debugging Field Guide

Gautam's Personal Bug Bible

Windows · Expo SDK 54 · Android Emulator

This guide documents every single error encountered while setting up and learning React Native — from installing Chocolatey to native module crashes. Each bug is explained in plain language: what it is, why it happened, how we fixed it, exact commands, and how to prevent it next time.

Error #	Error Name	Severity	Source
#01	No Android Device / Emulator Found	HIGH	Chat 1 & 2
#02	Emulator: System UI Isn't Responding	LOW	Chat 1 & 2
#03	Expo Go SDK Version Mismatch	HIGH	Chat 1 & 2
#04	ERR_PACKAGE_PATH_NOT_EXPORTED (Metro)	MEDIUM	Chat 1
#05	Wrong Directory When Running Commands	LOW	Chat 1
#06	Missing app/(tabs)/_layout.tsx	MEDIUM	Chat 3
#07	Java ClassNotFoundException / NoClassDefFoundError	HIGH	Chat 4 & 5

#08	Stale Gradle Build Cache	HIGH	Chat 5
#09	Canary Package Breaking Android Build	HIGH	Chat 4 & 5
#10	npm warn deprecated / Vulnerability Warnings	LOW	Chat 1

Quick Glossary

Every technical term used in this guide is defined here. Read this first — it makes all the error explanations 10x clearer.

Expo	A framework built on top of React Native. Makes starting RN projects way easier — it pre-configures bundling, dev tools, and a standard project structure. Think of it as a powered-up starter kit.
Expo SDK	The version number of Expo (e.g. SDK 54). Each SDK version locks compatible versions of React Native, Metro, and all dependent packages. Mixing SDK versions = crashes.
Expo Go	A pre-built app you install from the Play Store. Lets you run your dev app by scanning a QR code. Limitation: only supports official Expo packages. Custom native packages won't work inside it.
expo-dev-client	Your personal version of Expo Go, custom-built for your project. Compiled once with your native packages — gives you the same fast QR workflow but with full native module support.
Metro Bundler	React Native's JS bundler. Takes all your .tsx/.js files, combines them into one bundle, and serves it to your phone/emulator in real time. Fast Refresh = auto-update on file save.
Gradle	Android's build system. When you run npx expo run:android, Expo calls Gradle to compile all native Java/Kotlin code and produce the final .apk file. gradlew is the Gradle wrapper script.
Gradle Build Cache	Gradle saves compiled files to speed up future builds. Stale (outdated) cache after package changes causes crashes. gradlew clean wipes this cache and forces a clean rebuild.
APK	Android Package (.apk). The actual installable app file for Android. Like a .exe but for Android devices and emulators.
Native Module	Code written in Java/Kotlin (Android) or Swift (iOS) that does things JavaScript can't do directly — microphone, Bluetooth, camera, etc. Requires a full rebuild when changed.
node_modules	A folder containing ALL JavaScript packages your project depends on. Can be 500MB+. Safe to delete anytime — run npm install to regenerate from package.json.
package.json	The dependency manifest of your project. Lists every package and its version. npm install reads this and downloads everything listed.
ADB	Android Debug Bridge. CLI tool for communicating with Android devices/emulators. Used for installing APKs, reading logs, and checking connections.

AVD	Android Virtual Device. The emulator. A fake Android phone running inside your PC via Android Studio.
ANDROID_HOME	Environment variable pointing to where your Android SDK is installed. Without it, ADB and Gradle build tools don't work.
Canary Package	Unstable, experimental version of a package, marked with -canary in the version string. Like an alpha build. Often broken on Android even when working on iOS.
Stack Navigator	Navigation type where screens stack on top of each other like cards. Going back removes the top card.
Tab Navigator	Navigation type with a tab bar at the bottom. Used for switching between app sections like Home, Profile, Settings.

Error #01

■ HIGH

No Android Device / Emulator Found

Source: Chat 1 & 2

What is this error?

When running `npm run android` or `npx expo start --android`, the terminal printed:

```
CommandError: No Android connected device found,
and no emulators could be started automatically.
Connect a device or create an emulator
# https://docs.expo.dev/workflow/android-studio-emulator
```

Why did it happen?

Expo needs somewhere to run your app. It checks two places: (1) a connected physical phone via USB, and (2) a running Android emulator (AVD). At the time this error hit, no emulator had been created in Android Studio and no phone was connected. Expo gave up.

How we fixed it

We created a Pixel 8 AVD in Android Studio and downloaded the API 35 system image:

- Open Android Studio
- More Actions → Virtual Device Manager
- Create Device → select **Pixel 8** → Next
- Download the API 35 Google Play Intel x86_64 system image (~1.6 GB)
- Select it → Finish → press ■ to start the emulator
- Wait for the Android home screen to appear BEFORE running your app

Commands used

```
# After emulator is fully booted:
$ npm run android
# OR
$ npx expo start
# Then press 'a' to open on Android
```

■■ Data usage: Downloading the API 35 system image costs ~1.6 GB. Do this on WiFi ONLY.

■■ How to Prevent This Next Time

■ Always create your AVD BEFORE trying to run your project for the first time.

■ Start the emulator from Android Studio first — let it fully boot to the home screen THEN run your app.

■ On limited data? Use your physical phone with Expo Go — zero big downloads needed.

■ Run adb devices in terminal to check if any device is connected before running your app.

Error #02

■ LOW

Emulator: 'System UI Isn't Responding'

Source: Chat 1 & 2

What is this error?

After the Pixel 8 emulator started, a popup appeared on the screen saying **'System UI isn't responding'** with two buttons: **Wait** and **Close App**. It kept appearing repeatedly.

Why did it happen?

Android's System UI manages the home screen, status bar, and notifications. When the emulator boots, it runs an entire virtual Android phone inside your PC — extremely CPU and RAM heavy. On lower-end machines or during the cold first boot, Android panics because System UI takes too long to respond. **This is NOT a bug in your code.**

Cause	Explanation
`Slow cold boot`	First emulator start is always the heaviest — everything loading from scratch
`Low RAM allocated`	Default AVD RAM (2048 MB) is often not enough
`No hardware acceleration`	Without HAXM/WHPX, emulator runs on CPU only — brutally slow

How we fixed it

In our case: click **Wait** and wait 2–3 minutes. It resolved itself. For persistent cases, here are additional fixes:

- Always click **WAIT** — never Close App (that kills the emulator)
- Increase AVD RAM: Device Manager → Edit Pixel 8 → Show Advanced Settings → set RAM to 4096 MB
- Verify hardware acceleration: File → Settings → SDK Manager → SDK Tools → confirm Android Emulator Hypervisor Driver ■
- Try Cold Boot: Device Manager → dropdown on Pixel 8 → Cold Boot Now

> ■■ **The emulator can use 2–4 GB of RAM from your PC. Close all heavy apps before starting it.**

■■ How to Prevent This Next Time

- ■ Always wait 2–3 minutes after the emulator starts before running your app.
- ■ Set AVD RAM to at least 4096 MB when creating the emulator initially.
- ■ For smoother development, use a physical phone — it uses zero PC RAM and responds faster.
- ■ Don't run heavy apps (games, videos) in the background while the emulator is open.

Error #03

■ HIGH

Expo Go SDK Version Mismatch

Source: Chat 1 & 2

What is this error?

Two versions of this error appeared:

- **Version A:** 'Project is incompatible with this version of Expo Go. The installed version of Expo Go is for SDK 55. The project uses SDK 54.'
- **Version B:** 'This project requires a newer version of Expo Go. Download the latest version from the Play Store.'

Why did it happen?

Expo Go is a pre-built container compiled specifically for one SDK version. SDK 54 and SDK 55 have different native code inside. Mixing them is like trying to play a PS5 game on a PS4 — the hardware doesn't match.

- Emulator had Expo Go SDK 55 pre-installed, but the project was SDK 54
- Phone had an older Expo Go, but the project had been upgraded to a newer SDK

How we fixed it — Version A (Emulator)

Manually downloaded the correct Expo Go APK (SDK 54) and installed it on the emulator:

```
# 1. Download the correct APK from:
# https://expo.dev/go?sdkVersion=54&platform=android&device=false

# 2. Drag the downloaded .apk file directly onto the emulator window
# 3. Click Install when prompted on the emulator screen
```

How we fixed it — Version B (Project / SDK conflict)

Deleted the broken project and created a fresh one with a compatible SDK version:

```
# Step 1: Navigate out of the project
$ cd ..

# Step 2: Delete the broken project
$ Remove-Item -Recurse -Force berealclone

# Step 3: Create fresh compatible project
$ npx create-expo-app@3 berealclone

# Step 4: Start it
$ cd berealclone
$ npx expo start
```

■■ Data: Downloading a specific Expo Go APK ~80–100 MB. Creating a new project ~50–80 MB.

■■ How to Prevent This Next Time

- ■ Always check your project SDK version first: cat package.json (look for the 'expo' line)
- ■ Match your project SDK with your Expo Go version BEFORE starting development.
- ■ When creating a new project, always specify a stable version: npx create-expo-app@3
- ■ Don't accept Expo upgrade prompts blindly — check compatibility first.
- ■ When the emulator asks to install a recommended Expo Go version, press Y.

Error #04

■ MEDIUM

ERR_PACKAGE_PATH_NOT_EXPORTED (Metro)

Source: Chat 1

What is this error?

```
Error [ERR_PACKAGE_PATH_NOT_EXPORTED]:
Package subpath './src/lib/TerminalReporter'
is not defined by "exports" in
...\node_modules\metro\package.json
```

Appeared when running `npx expo start` after downgrading from Expo SDK 55 to SDK 53.

Why did it happen?

A package version mismatch. Here's the chain of events:

- Project was originally created with Expo SDK 55
- We downgraded ONLY Expo: `npx expo install expo@~53.0.0`
- All other packages (metro, react-native, etc.) stayed at their SDK 55 versions
- Metro v0.81.x (SDK 55) exposes its file paths differently than what Expo 53 expects
- Expo 53's startup code tried to access `'./src/lib/TerminalReporter'` — a path Metro 0.81 no longer exports publicly

Analogy: you swapped the band leader to an older version, but kept band members hired by the new leader. They don't know the old leader's signals.

How we fixed it

```
# Step 1: Delete all node_modules (no internet needed)
$ Remove-Item -Recurse -Force node_modules

# Step 2: Reinstall everything at correct compatible versions
$ npm install
```

This forced npm to resolve ALL packages at versions compatible with Expo 53, not just the Expo package itself.

■■ **Data: npm install after a full node_modules delete = ~100–150 MB.**

■■ How to Prevent This Next Time

- ■ When downgrading OR upgrading Expo, NEVER change just the expo package alone.
- ■ Use: npx expo install expo@VERSION --fix — the --fix flag updates all dependent packages automatically.
- ■ After any major version change, always delete node_modules and run npm install fresh.
- ■ Use npx expo-doctor to check for package compatibility issues.

Error #05

■ LOW

Wrong Directory When Running Commands

Source: Chat 1

What is this error?

```
PS C:\Users\Win\Desktop\Coding_Stuff\React_Native> npx expo start
Need to install the following packages:
expo@55.0.11
Ok to proceed? (y)
```

Instead of starting the project, it asked to install Expo from scratch — a sign that something is fundamentally wrong with the context.

Why did it happen?

`npx expo start` looks for an Expo project in the CURRENT directory. If no project is found (no package.json with expo listed), it assumes you want to create a new one and tries to install the latest Expo. The terminal was in the `React_Native` parent folder, NOT inside the `berealclone` subfolder.

How we fixed it

```
# Press N then Enter to cancel the install prompt

# Check where you actually are
$ pwd

# Navigate INTO your project
$ cd berealclone

# Now run the command again
$ npx expo start
```

How to read your terminal prompt

```
# WRONG — you're in the parent folder:
PS C:\...\React_Native>

# CORRECT — you're inside your project:
PS C:\...\React_Native\berealclone>
```

■■ How to Prevent This Next Time

- ■ Before EVERY command, check your terminal prompt — it shows your exact location.
- ■ Your prompt must end with \berealclone> before running any npx expo or npm commands.
- ■ Use pwd (print working directory) whenever confused about your location.
- ■ VS Code's integrated terminal (Ctrl + `) opens inside your project folder by default — use it.

Error #06

■ MEDIUM

Missing app/(tabs)/_layout.tsx

Source: Chat 3

What is this error?

The app was running and showing content, but **no tab bar appeared at the bottom**. Only one screen was visible with no way to navigate to About or Profile tabs.

Why did it happen?

In Expo Router, every navigation group folder needs its own `_layout.tsx` file. The `app/(tabs)` folder was missing this file entirely.

File	What it does
`app/_layout.tsx`	Root layout — defines the main Stack navigator for the whole app. Must exist at app/ level. NEVER move this.
`app/(tabs)/_layout.tsx`	Tabs layout — defines the Tab navigator with the bottom tab bar. Must be a SEPARATE file inside the (tabs) folder.

How we fixed it

Created a new file at `app/(tabs)/_layout.tsx`:

```
import { Tabs } from 'expo-router';

export default function TabsLayout() {
return (
<Tabs>
<Tabs.Screen name='index' options={{ title: 'Home' }} />
<Tabs.Screen name='about' options={{ title: 'About' }} />
<Tabs.Screen name='profile' options={{ title: 'Profile' }} />
</Tabs>
);
}
```

After saving, the tab bar appeared immediately at the bottom.

■■ Critical Warning

Do NOT move `app/_layout.tsx` into tabs. You need BOTH files simultaneously. Moving the root layout breaks the entire app.

Correct file structure

```
app/
_layout.tsx # Stack navigator (ROOT — never move this)
(tabs)/
_layout.tsx # Tabs navigator (CREATE this as a new file)
index.tsx
about.tsx
profile.tsx
```

■■ How to Prevent This Next Time

- ■ Every navigation group folder in Expo Router needs its own _layout.tsx — non-negotiable.
- ■ Never move existing _layout.tsx files — create new ones in subfolders.
- ■ Can't see tab bar? FIRST check if app/(tabs)/_layout.tsx exists.
- ■ The root _layout.tsx and the tabs _layout.tsx are two SEPARATE files that must both exist.

Error #07

■ HIGH

Java ClassNotFoundException / NoClassDefFoundError

Source: Chat 4 & 5

What is this error?

```
java.lang.NoClassDefFoundError:
Failed resolution of: Lexpo/modules/ui/ExpoUIModule

Caused by: java.lang.ClassNotFoundException:
Didn't find class 'expo.modules.ui.ExpoUIModule'
```

App crashed immediately on launch after installing `@expo/ui` and running `npx expo start`.

Why did it happen?

This is a Java runtime error, not a JavaScript error. Plain English: *'I was compiled expecting this Java class to exist, but at runtime it's nowhere to be found.'*

Root cause — two problems combined:

- `@expo/ui` was installed as npm package (JS side knew about it)
- **But** `npx expo start` was used instead of `npx expo run:android`
- `npx expo start` only serves JavaScript via Metro — it does NOT compile native code
- So the APK had NO ExpoUI Java code compiled into it
- When JS tried to call the native module, the Java class simply didn't exist in the APK

Command	What it actually does
`npx expo start`	Serves JavaScript only via Metro Bundler. Fast. No native rebuild. Use for JS/UI changes.
`npx expo run:android`	Compiles ALL native code + JS, builds a real APK. Slow. Use after any native package change.

How we fixed it

```
# Step 1: Uninstall @expo/ui
$ npm uninstall @expo/ui

# Step 2: Clean stale Gradle cache
$ cd android
$ .\gradlew clean
$ cd ..

# Step 3: Replace @expo/ui code in index.tsx with standard RN
# BEFORE (broken):
import { Button, Host } from '@expo/ui/jetpack-compose';

# AFTER (working):
import { View, TouchableOpacity, Text } from 'react-native';

# Step 4: Full native rebuild
$ npx expo run:android
```

■■ Data: gradlew clean + npx expo run:android after clean rebuild = 200–500 MB. WiFi only!

■■ How to Prevent This Next Time

■ RULE: Use npx expo run:android after installing ANY native package. npx expo start = JS changes only.

■ After installing a native package, always rebuild. There is no shortcut. This is non-negotiable.

■ The difference: npx expo start = JavaScript only. npx expo run:android = full native + JS rebuild.

■ Check if a package has 'canary' or 'alpha' in the version before using it in your project.

Error #08

■ HIGH

Stale Gradle Build Cache

Source: Chat 5

What is this error?

After uninstalling `@expo/ui` and running `npx expo run:android`, the SAME NoClassDefFoundError crashed the app again — as if nothing had been fixed.

Why did it happen?

Gradle caches compiled build files inside `android/app/build` to speed up future builds. When `@expo/ui` was installed, Gradle compiled its Java code into the cache. When we uninstalled it from npm, the cached `.class` files were NOT automatically deleted.

So on the next build, Gradle said *'oh I already compiled this, I'll use the cached version'* — except the cached version still referenced the removed module. The stale cache ghosts your build even after you fix the actual problem.

How we fixed it

```
# Step 1: Navigate into the android subfolder
$ cd C:\Users\Win\Desktop\Coding_Stuff\React_Native\berealclone\android

# Step 2: Wipe ALL Gradle build cache
$ .\gradlew clean
# Expected output: BUILD SUCCESSFUL in Xs

# Step 3: Go back to project root
$ cd ..

# Step 4: Clean rebuild
$ npx expo run:android
```

The golden rule

ANY time you install, uninstall, or upgrade a native package — run `gradlew clean` before rebuilding. No exceptions.

```
# The holy trinity after any native package change:
$ npm uninstall PACKAGE_NAME # (or npm install for new package)
$ cd android && .\gradlew clean && cd ..
$ npx expo run:android
```

■■ Data: gradlew clean uses almost zero data (deletes local files only). npx expo run:android = 200–500 MB.

■■ How to Prevent This Next Time

- ■ After ANY native package change, always run gradlew clean before rebuilding.
- ■ gradlew clean only deletes files in android/app/build — it never touches your source code.
- ■ Same crash after a fix? Assume stale Gradle cache. Run gradlew clean immediately.
- ■ You can run gradlew clean as many times as you want — it's completely safe.

Error #09

■ HIGH

Canary Package Breaking Android Build

Source: Chat 4 & 5

What is this error?

The tutorial instructed installing `@expo/ui`. After installing and rebuilding, the app crashed with ClassNotFoundException. Even after clean rebuilds, the package kept causing issues specifically on Android.

```
# During gradlew clean, this appeared in the output:
- [canary] expo-ui (0.2.0-canary-20260121-a63c0dd)
# ^^^^^^^
# This 'canary' tag is the red flag
```

What is a canary package?

A canary release is an unstable, experimental build of a package — even earlier than alpha or beta. The name comes from 'canary in a coal mine' — it's pushed out to detect problems early, often before the package is fully functional.

- Marked with `-canary` + a date in the version string
- The tutorial author was on macOS/iOS where the canary happened to work
- Android support in canary builds is frequently broken or incomplete
- The package was experimental and not ready for production Android use

How we fixed it

Uninstalled the canary package entirely and replaced its components with stable React Native equivalents:

```
# Step 1: Uninstall the canary package
$ npm uninstall @expo/ui

# Step 2: Replace in your code
# BEFORE — using @expo/ui:
import { Button, Host } from '@expo/ui/jetpack-compose';
// <Host><Button onPress={...} /></Host>

# AFTER — using stable React Native:
import { View, TouchableOpacity, Text } from 'react-native';
// <TouchableOpacity onPress={...}><Text>Click me</Text></TouchableOpacity>

# Step 3: Clean + rebuild
$ cd android && .\gradlew clean && cd ..
$ npx expo run:android
```

■■ When a tutorial uses canary packages and you're on Android, cross-reference with the package's GitHub issues page for Android-specific known bugs.

■■ How to Prevent This Next Time

- ■ Before installing any package, check if the version contains 'canary', 'alpha', or 'nightly'.
- ■ Canary packages are fine for iOS experiments — treat them as potentially broken on Android.
- ■ Check the package's GitHub Issues page for 'Android' problems before adopting it.
- ■ Prefer stable package alternatives over experimental ones during the learning phase.
- ■ npx expo-doctor will flag incompatible or experimental packages in your project.

Error #10

■ LOW

npm warn deprecated / Vulnerability Warnings

Source: Chat 1

What is this error?

```
npm warn deprecated glob@10.5.0: Old versions of glob are not supported...
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer supported
npm warn deprecated inflight@1.0.6: This module is not supported and leaks memory.

1 high severity vulnerability
To address all issues, run: npm audit fix
```

Why did it happen?

These warnings appear when packages your project depends on (or packages that your packages depend on) are outdated. The package maintainers added deprecation notices in newer npm versions, so npm now reports them every time you install anything.

The **'1 high severity vulnerability'** sounds alarming. Honest reality check:

- In a **learning project**, these won't affect you at all
- They only matter in production apps handling real user data, financial info, etc.
- These come from deeply nested transitive dependencies — packages you didn't even choose directly

What you should and shouldn't do

Action	Verdict
`Ignore the warnings during learning`	■ Correct — focus on learning, not chasing warnings
`Run npm audit fix blindly`	■ Wrong — can break package compatibility
`Read npm audit output`	■ Fine — educational to understand what's affected
`Panic and delete everything`	■ Absolutely not. These warnings don't crash your app

■■ NEVER run npm audit fix --force. It can silently downgrade packages and break your entire project in ways that are hard to debug.

■■ How to Prevent This Next Time

- ■ npm warn deprecated = informational only. Not an error. Not urgent. Not a crash.
- ■ 'high severity vulnerability' in a learning project = not your problem right now.
- ■ When building a real production app with user data, THEN address vulnerabilities properly.
- ■ Never run npm audit fix blindly — always read what it will change first.

Quick Cheat Sheet

Print this. Stick it on your monitor. Save your sanity.

What you want to do	Command	Notes
Start dev server (JS only)	`$ npx expo start`	For JS/UI changes only. No native rebuild.
Open on Android (after start)	`$ npx expo start → press a`	Launches on connected emulator/device.
Full native rebuild	`$ npx expo run:android`	After ANY native package install/change.
Clean Gradle cache	`$ cd android && .\gradlew clean && cd ..`	Run before rebuilding after native changes.
Install Expo-safe package	`$ npx expo install PACKAGE`	Not npm install — Expo picks the right version.
Uninstall package	`$ npm uninstall PACKAGE`	Then: gradlew clean + run:android.
Delete node_modules	`$ Remove-Item -Recurse -Force node_modules`	PowerShell only. Then run npm install.
Reinstall all packages	`$ npm install`	After deleting node_modules.
Check current directory	`$ pwd`	Always know where you are.
Enter project folder	`$ cd berealclone`	Must be in project root for all commands.
Go up one folder	`$ cd ..`	Navigate to parent directory.
Check ADB connected devices	`$ adb devices`	See if emulator/phone is detected.
Check project Expo version	`$ cat package.json`	Look for the 'expo' line.
Clear terminal (Windows)	`$ cls`	PowerShell / cmd.

■ The 8 Golden Rules

1. npx expo start = JavaScript changes only. npx expo run:android = native changes. Know the difference.

2. Native package changed? Always: (1) gradlew clean → (2) npx expo run:android. No shortcuts.

3. Check your package.json Expo SDK version BEFORE running anything.

4. Same error after a fix? Stale Gradle cache. Run gradlew clean immediately.

5. npm warn deprecated = harmless noise. Ignore it during learning.

6. Always be inside your project folder (\berealclone) before running commands.

7. WiFi before any download. Your mobile data is precious and non-negotiable.

8. Check for 'canary' or 'alpha' in a package version before using it in Android projects.

Built from real pain. Every error in this guide was hit face-first. Now go build EarGuard. ■